

Roof Project Root Specification

Project Roof

Data Foundation Lab
DFLab FHO Pty Ltd – Proprietary Limited Company
(13 676 090 289)

Project Vision

Roof aims to give individuals control over their personal data, applications and digital interactions through local-first, user-operated computing. Its current direction is software-first and hardware-independent: users should be able to operate Roof on supported devices they already own, with dedicated hardware as an optional deployment model. This is a development target; the existing implementation remains rooted in the Raspberry Pi 5 dedicated Linux appliance.

Project End Goal

The long-term goal is a Personal Internet: interconnected, user-operated environments in which individuals retain control of their data and can communicate, share information and run applications on their own infrastructure. Decentralisation, federation and interoperability guide this vision. Universal reach, resilient availability and a broad application ecosystem are aspirations, not guarantees of continuous connectivity or capabilities delivered by the initial developer preview.

Project Background

Roof began in 2024 from Data Foundation Lab's internal Federated Database (FDB) initiative. The central problem was how to retain personal control of data while keeping it accessible when a person's everyday device was switched off or disconnected. The first-ever model specification proposed a Raspberry Pi Zero 2 W powered from a modem/router's USB port and connected over Wi-Fi. Open community collaboration was already part of that founding concept.

The early design explored distributed storage approaches, including blockchain-based distribution; later project documents also discussed IPFS integration. The project chose not to depend on broad dispersal of personal data across third-party infrastructure. This records Roof's design rationale, rather than a claim that all distributed storage systems have the same privacy properties.

The resulting at-home concept placed storage under the user's own roof. Inspired by household modems and routers, it sought sustained availability while remaining dependent on power, connectivity and functioning hardware. Subsequent specifications and development adopted Raspberry Pi 5 dedicated hardware, with Roof Engine providing the node runtime and RoofDeck the user interface.

In October 2026, planned hardware Beta Round 3 was blocked by Raspberry Pi 5 procurement costs and availability amid wider hardware supply constraints caused by global AI data centre and compute demands. This prompted the decision to transition development to open source, bringing the long-term plan forward unexpectedly. The software-first direction now seeks to separate Roof's core functionality from mandatory appliance ownership, preserving existing work while making dedicated hardware optional. Source publication and hardware independence are separate milestones.

Table of Contents

Project Vision *1*

Project End Goal *1*

Project Background *1*

1. Project Objectives *5*

1.1 Personal Data Ownership 5

1.2. Decentralisation and Federation 5

1.3. Continuous Data Availability 5

1.4. Data Privacy and Security 5

1.5. Scalable and Interoperable Infrastructure 5

1.6. User-Centric Experience 6

1.7. Open Developer Community Ecosystem 6

2. Project Phases and Evolution *6*

2.1 Historical Internal Availability and Hardware Beta 6

2.2 Historical Expansion Phases 6

2.3 October 2026 Open-Source Decision 6

2.4 Planned Developer Preview 6

2.5 Hardware-Independent Development Direction 6

2.6 Long-Term Evolution 7

3. Scope Definition *7*

3.1 In Scope 7

3.2 Out of Scope 7

3.3 Scope Ruling 7

4. Stakeholders & Users *8*

4.1 End Users 8

4.2 Developers 8

4.3 Internal Team 8

4.4 Beta Testers 8

5. Use Cases & User Flows *8*

5.1 First-Time Setup 9

5.2 File Sync Flow 9

5.3 Remote Access Flow 9

5.4 Failure Recovery Flow 9

6. System Overview *9*

6.1 Capability and Maturity Overview 10

7. System Architecture *10*
7.1 Hardware **10**
7.2 Software **11**
7.3. Logical Architectural Model **11**
7.3.1. Architectural Principles 11
7.3.2. Core Architectural Components 12
7.3.3. Communication Model 14
7.3.4. Dependency and Layering Model 14
7.3.5. Platform Scope 15
7.3.6. Decommissioned Components 15
7.3.7. High-Level System Relationships 16
8. Component Boundaries & Contracts *16*
8.1 Roof Engine **16**
8.2 RoofDeck **16**
8.3 Roof PathFinder **16**
8.4 Roof Client Library **17**
8.5 Roof Common **17**
9. Networking Specification *17*
9.1 Network Environments **17**
9.2 Network Behaviour Principles **17**
10. Security and Data Privacy Model *18*
10.1 Core Principles **18**
11. System Constraints & Assumptions *19*
11.1 System Constraints **19**
11.2 System Assumptions **19**
12. Risks & Limitations *20*
12.1 Identified Risks **20**
12.2 System Limitations **21**
13. Roadmap & Future Phases *21*
13.1 Current Direction **21**
13.2 Preview Scope **22**
13.3 Entry Criteria **22**
13.4 Exit Criteria **22**
13.5 Phase Transition Rules **22**
13.6 Future Considerations **22**
14. Compliance & Legal *22*
14.1 Compliance Scope **22**
14.2 Data Privacy Compliance **23**

14.3 Jurisdiction Considerations **23**
14.4 Roles & Responsibilities **23**
14.5 Legal Boundaries of the System **23**
14.6 Future Regulatory Alignment **23**
15. Human Interventions and Interactions ***23***
Content Filtering and Moderation **23**
User Verification and Authentication **24**
Reporting and Response Mechanisms **24**
Data Privacy and Consent **24**
Community Guidelines **24**
Legal Compliance **24**
Technological Safeguards Against Modern Innovations **24**
Appendices ***25***
Glossary **25**
Acronyms **25**
Diagrams **25**
References and Document History **25**

Document revision: 9 November 2026 (supersedes the 23 April 2026 root revision)
Status: Living root specification. Open-source transition; preview planned
Owner: Steve Yoo
Last Updated: 09/11/2026

Purpose: Preserve Roof's inception and evolution, define its guiding concepts and architectural boundaries, and direct detailed requirements to component specifications. Statements of intent are distinct from implemented capabilities and release support.

---

1. Project Objectives

These objectives guide component requirements and release acceptance criteria. They are design commitments, not a declaration that every objective has been achieved. Component specifications define measurable requirements; release notes record validated behaviour.

1.1 Personal Data Ownership

Enable users to fully control and store their data on their own devices, removing reliance on centralised cloud providers.

1.2. Decentralisation and Federation

Develop interoperable, independently controlled nodes that can communicate and share data with user authorisation. Federation does not itself provide replication, backup or protection from a single-node failure.

1.3. Continuous Data Availability

Improve availability through user-managed infrastructure and resilient operation. Access remains dependent on the host, power, storage and the relevant network path; continuous availability is not guaranteed.

1.4. Data Privacy and Security

Protect privacy through user-controlled storage, explicit access decisions and secure communication. Local ownership reduces dependence on third-party custody but does not eliminate software, host-security or operational risks.

1.5. Scalable and Interoperable Infrastructure

Build an architecture that supports additional applications and services on top of the protocol, facilitating seamless communication and interoperability between users' personal devices and Roof nodes.

1.6. User-Centric Experience
Ensure that the system is easy to install, manage, and integrate with existing personal devices (phones, laptops, etc.), making the transition from traditional cloud storage to a decentralised model both simple and effective.

1.7. Open Developer Community Ecosystem
Create a collaborative, open-source developer community that drives innovation, enhances security, and adapts the technology to meet evolving user needs.

2. Project Phases and Evolution
2.1 Historical Internal Availability and Hardware Beta
The former NV1 Internal Availability phase described file storage and management, a remote-access client, local and remote access, and synchronisation. The subsequent hardware Beta Test phase focused on Roof application integration and controlled evaluation.

These labels preserve the earlier plan, not a claim that every listed capability or acceptance criterion was completed.

2.2 Historical Expansion Phases
The earlier sequence envisaged Public Availability, Business Availability, a Developer Availability platform and Universal Availability. Node-to-node communication, an application ecosystem, global connectivity and community governance were associated with that plan.

The sequence is retained as history and is superseded as a delivery schedule by the public development roadmap.

2.3 October 2026 Open-Source Decision
Planned hardware Beta Round 3 could not proceed because of Raspberry Pi 5 procurement costs and availability amid wider hardware supply constraints. In October 2026, DFLab decided to transition to open-source development. The blocked hardware round is distinct from future software beta testing.

2.4 Planned Developer Preview
Developer Preview 0.1.0 is planned for 16 November 2026 as an experimental, source-only release of the existing implementation for inspection, supported source builds and contribution. It is not a consumer beta, a package of executable installers or a completed hardware-independent runtime.

2.5 Hardware-Independent Development Direction
The next direction is to separate the existing core from dedicated-host assumptions, demonstrate it on a generic Linux host, and validate identity, storage, networking, application and sharing workflows. Safe software-first installation and optional appliance deployment follow their own readiness criteria.

2.6 Long-Term Evolution

Broader platform support, a mature application ecosystem and resilient Personal Internet infrastructure remain future goals. The proposed Buddy System is deferred research into availability and recovery; it is not a current resilience guarantee.

3. Scope Definition

This root specification records Roof's history, purpose, objectives and architectural boundaries. It guides the ecosystem without prescribing exhaustive implementation details.

Scope labels should be read with the applicable component and release context; inclusion in a specification does not establish implementation or support.

NOT CONSIDERED / OUT-OF-SPEC: Excluded from the stated document or release scope. An implementation detail outside this root document may still belong within a component specification.

CONSIDERED / IN-SPEC: A defined requirement or design responsibility within the stated scope. Delivery and validation status must be established separately.

TO BE CONSIDERED / REVISIT IN THE FUTURE: A direction reserved for later assessment, not a current delivery commitment.

3.1 In Scope

- Roof ecosystem architecture
- Component roles and interactions
- Network and communication model
- Security principles

3.2 Out of Scope

- Internal implementation of:
  - Roof Engine
  - RoofDeck
  - Roof PathFinder
- UI/UX behaviour
- API-level implementation details

3.3 Scope Ruling

This document does **NOT** define:

- Internal logic of components, apps, APIs, etc.
- UI/UX behaviour of RoofDeck
- API definitions
- SDK implementations

If it answers "**how**", it should be defined in their respective specification document.

If it answers "**what/why/boundaries**", it should be defined in this root specification.

4. Stakeholders & Users

These groups describe the intended ecosystem. The initial source-only preview is for developers and technical evaluators; it does not establish readiness for the broader end-user audience.

4.1 End Users

End users are the intended beneficiaries: individuals operating personal Roof environments and controlling data and sharing permissions.

The long-term experience should be understandable and reliable for both non-technical and advanced users. Users remain responsible for host availability, credentials and backups.

4.2 Developers

Developers build and maintain Roof components, applications and integrations. They need documented boundaries, supported build environments and contribution processes, and must respect user authorisation and privacy.

The source-only preview initially serves this technical audience.

4.3 Internal Team

DFLab is the project's originating team and custodian of its architecture, specifications and release direction. Open contribution does not itself transfer governance or imply that future community-governance arrangements are already established.

4.4 Beta Testers

Beta testers evaluate defined pre-release workflows and report usability, reliability and security issues under documented conditions. Historical hardware beta participation is separate from developer-preview evaluation and future software beta programmes.

5. Use Cases & User Flows

These representative flows express intended user outcomes. Their detailed steps, supported environments and acceptance tests belong in the Engine, RoofDeck and networking specifications.

Existing appliance workflows must be distinguished from future software-first onboarding.

The preview's source-build guidance defines the technical evaluation path; it is not a consumer installation flow.

5.1 First-Time Setup

Historical appliance onboarding used a Roof-hosted Wi-Fi access point: power on the device, connect through RoofDeck, register and configure networking. This remains a deployment-specific workflow.

The software-first target is installation and configuration on a supported host, establishment of a persistent Roof identity, and connection through RoofDeck.

AP provisioning and whole-host administration are optional appliance functions, not universal onboarding requirements.

5.2 File Sync Flow

The intended flow detects an authorised file change, transfers the required data, validates synchronisation state and handles conflicts under component-defined rules.

Supported directions, conflict behaviour and recovery must be stated per release; this root does not promise complete synchronisation across every device or environment.

5.3 Remote Access Flow

A client locates its node, establishes authenticated communication, and performs authorised operations. PathFinder may assist when a local path is unavailable.

Remote operation depends on endpoint and network availability, including any required relay infrastructure.

5.4 Failure Recovery Flow

When connectivity, a host or an operation fails, the system should surface the failure, preserve data integrity and support reconnection or recovery.

Automatic and user-initiated recovery responsibilities belong in the component specifications. Synchronisation is not a substitute for user-managed backup.

6. System Overview

This overview separates the existing foundation, the planned public preview and future development. Architectural responsibilities describe what components are for; they do not certify every workflow, platform or security property.

The existing dedicated-hardware implementation includes foundational identity, communication, storage, networking, application integration and data-sharing functionality.

Those capabilities must be preserved and validated as Roof moves away from dedicated Linux host assumptions. Their exact supported behaviour and limitations belong in component documentation and release notes.

6.1 Capability and Maturity Overview

Existing foundation: identity and authentication, communication and command exchange, local data operations, networking, RApp integration and data-sharing functionality are established within the dedicated-hardware implementation. The transition must validate their supported behaviour on independent hosts.

Planned preview: Developer Preview 0.1.0 exposes approved source and development guidance for technical evaluation. It does not add a completed portable Engine, consumer installers or a supported everyday-device workflow.

Future goals: software-first operation on verified hosts, wider interoperability, an expanded application ecosystem and improved resilience. Hardware independence, operating-system portability and end-user readiness are separate achievements.

7. System Architecture

The five core software components are Roof Engine, RoofDeck, Roof PathFinder, Roof Client Library and Roof Common.

Roof Native Apps (RApps) use the platform's application facilities; they are not a sixth core infrastructure layer.

The component roles below guide both the existing appliance implementation and the intended software-first evolution.

7.1 Hardware

Historically, Roof used dedicated single-board hardware, progressing from the initial Raspberry Pi Zero 2 W concept to Raspberry Pi 5 development.

Hardware independence now means separating the core runtime from a mandatory Raspberry Pi and from ownership of an entire host operating system. It does not mean that all devices or operating systems are supported.

The target deployment model should:

- Operate efficiently on supported general-purpose hosts as well as optional dedicated devices
- Separate hardware-specific provisioning and privileged device management from core runtime behaviour
- Enable appliance functions only within explicitly configured deployment profiles

Refer to the Roof Hardware Specification for dedicated-device requirements and to the Roof Engine Specification for runtime and host-integration boundaries.

Hardware availability must not define the long-term software architecture.

7.2 Software

Detailed requirements and implementation belong in the following component specifications. Historical filenames are retained as document identifiers; they do not imply that public editions are already available:

- Roof Engine Specification: **[INTERNAL] Roof Engine Project Specification_DataFoundationLab_2024-PRESENT.docx**
- Roof PathFinder Specification: **[INTERNAL] Roof PathFinder Project Specification_DataFoundationLab_2024-PRESENT.docx**
- Roof Internal Library Specification: **[INTERNAL] Roof Internal Library Project Specification_DataFoundationLab_2024-PRESENT.docx**
- RoofDeck Specification: **[INTERNAL] RoofDeck Project Specification_DataFoundationLab_2024-PRESENT.docx**

7.3. Logical Architectural Model

Roof's logical architecture prioritises local client-to-node operation, with remote assistance when endpoints cannot communicate locally. The existing implementation is appliance-oriented.

The target retains its functional boundaries while separating core identity, data, communication and application services from hardware-specific provisioning and privileged host administration.

Roof Engine, RoofDeck, Roof PathFinder, Roof Client Library and Roof Common form the architectural baseline. Their roles remain distinct through the transition.

Generic-host execution, safe installation and broader operating-system portability require separate implementation and validation.

7.3.1. Architectural Principles

The Roof architecture is governed by the following core principles:

- **Local-First Operation**. The Roof shall prioritise direct communication between a client device and its associated Roof node whenever both are reachable within the same local environment, including but not limited to routed local network, access point mode, and direct ethernet mode. Local connectivity is considered the primary operational path of the platform.

- **Remote Connectivity as Secondary Path**. Where local communication is unavailable, the Roof may utilise remote communication pathways through Roof PathFinder or equivalent network coordination components to establish and maintain encrypted node-to-node or client-to-node connectivity. Remote coordination is a fallback and extension mechanism, not the architectural centre of the system.

- **End-to-End Encryption by Design.** User payloads and sensitive endpoint interactions require protection between the intended endpoints. Relays must not be decryption

endpoints or persistent stores for user content. The security specification must distinguish payload encryption, transport protection, storage protection, keys and routing metadata; these requirements are not blanket claims of verified implementation.

- **Modular Separation of Responsibilities**. The architecture shall preserve clear functional boundaries between core system processing, client-facing interfaces, network coordination, shared reusable logic, and platform integration layers. This is intended to support maintainability, flexibility, and future extensibility.

- **Cross-Platform Client Consistency**. Client-side logic shall be abstracted into reusable libraries wherever possible to ensure consistency across supported desktop and mobile platforms and to minimise duplication of functional logic across official applications.

- **Lightweight and Resource-Conscious Design.** Preserve efficient operation while adapting the runtime for supported general-purpose hosts. Raspberry Pi appliance provisioning, host accounts, network configuration and other privileged administration must be isolated from ordinary software operation.

7.3.2. Core Architectural Components

Roof Engine

Roof Engine is the core node runtime, responsible for data and file operations, internal services, resource handling and system coordination.

The existing implementation targets Linux ARM64 on Raspberry Pi 5 and includes dedicated-host integration. The intended independent runtime retains these core responsibilities while isolating optional appliance management.

Generic Linux x86-64 is the first proposed independent-runtime target, not current compatibility.

Minimum hardware resources, installation privileges, storage isolation, service lifecycle and cryptographic behaviour are defined and tested in the Engine specification and release documentation.

Windows and macOS Engine support remain later evaluation targets.

RoofDeck

RoofDeck is the official client-facing management application of the Roof ecosystem. It is responsible for onboarding, installation, registration, connection management, and ongoing user interaction with a user's Roof instance.

RoofDeck provides the principal interface through which users discover and connect to their local Roof, manage credentials, install and update official Roof-native applications, and monitor system status.

RoofDeck is designed to facilitate secure communication between the client device and the Roof node while also providing diagnostic and troubleshooting support for setup and operational continuity. It is intended to operate across both desktop and mobile platforms.

Internally, RoofDeck is architected such that core functional logic remains decoupled from graphical interface logic, with client-side operational capabilities delegated to shared native libraries where appropriate.

Roof PathFinder
Roof PathFinder is the network coordination layer of the Roof ecosystem. It is a lightweight real-time routing and relay service responsible for assisting connectivity between Roof nodes and enabling peer-to-peer communication across non-local environments.

PathFinder manages responsibilities such as node presence announcement, connection health signalling, routing assistance, and encrypted payload forwarding.

PathFinder's architectural role is routing and connection assistance, without ownership, decryption or persistent storage of user payloads.

"Stateless" refers to the absence of persistent payload custody, not the absence of transient presence, connection or routing state.

Metadata handling and retention boundaries belong in the PathFinder specification.

Roof Client Library
Roof Client Library is the shared client-side integration layer of the Roof ecosystem. It provides the reusable functionality required for Roof-enabled applications to securely communicate with a user's Roof node and the broader Roof network.

This includes abstraction of lower-level functions such as authentication handling, encrypted communications, session establishment, node discovery, registration flows, and interaction with Roof Engine services.

The Client Library is intended to remain decoupled from graphical user interface implementations and acts as the native functional layer beneath official client applications such as RoofDeck.

By centralising client communication and integration logic, it enables consistent behaviour across desktop and mobile platforms, reduces duplicated implementation effort, and improves maintainability across the client ecosystem.

Roof Common
Roof Common is the foundational shared code layer used across multiple Roof applications and libraries. It contains reusable logic, shared utilities, common abstractions, protocol definitions, internal models, logging structures, networking primitives, configuration models, and cryptographic helper functionality that are required throughout the Roof ecosystem.

Roof Common exists to improve standardisation, reduce code duplication, and ensure interoperability between components such as Roof Engine, RoofDeck, Roof Client Library, and Roof PathFinder.

It supports architectural consistency by allowing multiple components to depend on a common baseline of internal definitions and supporting logic.

7.3.3. Communication Model

Local Mode. Local Mode is the primary communication model of the Roof architecture. In this mode, a client device running RoofDeck communicates directly with the user's Roof node through the local environment.

This may occur over routed local network, Roof-hosted access point mode, or direct ethernet connection. Local Mode is intended to operate without dependency on internet connectivity or external relay infrastructure.

This communication path is considered the preferred and sovereign path of the system, as it preserves direct client-to-node interaction and reduces reliance on intermediary infrastructure.

Local Mode shall support secure encrypted communication between the client and the Roof node.

Remote Mode. Remote Mode is the secondary communication model of the Roof architecture. It is used where direct local communication is unavailable or where a Roof node or client must communicate across external networks.

In this mode, connectivity may be assisted by Roof PathFinder or equivalent coordination infrastructure.

Remote Mode does not replace Local Mode, but extends system reach beyond the local environment.

All such communication is expected to remain end-to-end encrypted between endpoints, with coordination infrastructure functioning only as a routing and relay layer.

Relay and Routing Constraints. Any intermediary coordination or relay service within the Roof architecture shall be constrained to routing, connection assistance, session signalling, or availability coordination functions.

Such services shall not be treated as authoritative data owners, long-term storage providers, or trusted decryption endpoints by default.

7.3.4. Dependency and Layering Model

The Roof architecture follows a layered composition model:

- **User Interaction Layer**: RoofDeck
- **Client Integration Layer**: Roof Client Library

- **Core Node Processing Layer**: Roof Engine
- **Network Coordination Layer**: Roof PathFinder
- **Shared Foundation Layer**: Roof Common

This layered structure ensures that reusable logic is centralised into common libraries, that client logic is separated from interface presentation, and that network coordination remains distinct from both node execution and client experience.

The result is an architecture that supports clean maintenance boundaries and future expansion.

7.3.5. Platform Scope

Platform history and target directions must be read separately from verified release support:

- **Roof Engine:** existing Raspberry Pi 5 / Linux ARM64 baseline; generic Linux x86-64 is the first independent-runtime target
- **Roof PathFinder:** documented Debian Linux ARM64 / Raspberry Pi 5 baseline; deployment requirements remain component-specific
- **RoofDeck:** desktop client baseline; Windows, macOS, Linux, iOS and Android are platform directions, with verified support declared per release
- **Roof Client Library:** reusable client integration; compatibility follows tested client and library builds
- **Roof Common:** shared definitions and utilities; reuse does not by itself establish platform compatibility

Decommissioned or legacy components may be retained in documentation where necessary for historical context, but are not considered part of the active baseline architecture unless reinstated through formal revision.

7.3.6. Decommissioned Components

The following components are presently **decommissioned** and do **NOT** form part of the current active architectural baseline:

Roof PathFinder-Bridge

Previously intended for public-based routing or point-to-point peer-to-peer (through UDP hole-punching) establishment support, this component has been decommissioned following changes to the global routing model that now leverage multichannel encrypted WebSocket-based network communication.

It may be permanently retired or reintroduced in future only where justified by a revised architectural need.

Roof Client SDK

Previously introduced to support development convenience for internal Roof-related applications and official distributed applications, this component is currently decommissioned.

Its former responsibilities are expected to be absorbed by or superseded through the Roof Client Library and other shared ecosystem components where appropriate.

7.3.7. High-Level System Relationships

RoofDeck uses Roof Client Library to communicate with Roof Engine.

Local communication is preferred; remote paths may use Roof PathFinder for connection assistance and encrypted forwarding.

Roof Common supplies shared definitions and utilities across these components.

RApps use defined Engine and client interfaces.

In the target architecture, optional appliance management sits outside the portable core runtime.

8. Component Boundaries & Contracts

8.1 Roof Engine

- Responsibility: Core node processing, data management, system coordination
- Inputs: Client requests, local system state
- Outputs: Processed data, sync operations, system responses
- Required properties (subject to component validation):
    - Access and sharing governed by explicit user authorisation
    - Local-first core operation
    - Defined data-integrity, consistency and recovery behaviour

8.2 RoofDeck

- Responsibility: User interaction and system control interface
- Inputs: User commands
- Outputs: Requests to Roof Engine, system state display
- Required boundaries:
    - User data remains under user control
    - Presentation and interaction logic separated from reusable client functionality

8.3 Roof PathFinder

- Responsibility: Network coordination and routing assistance
- Inputs: Node connection requests
- Outputs: Routing signals, encrypted packet forwarding
- Required boundaries:
    - Transient routing and presence state only as defined by the PathFinder specification
    - No decryption or persistent storage of user payloads
    - No data ownership

8.4 Roof Client Library

- Responsibility: Reusable client authentication, session, discovery and Engine-integration functions, independent of graphical interfaces.

It's interfaces and compatibility requirements belong in the Roof Internal Library Specification.

8.5 Roof Common

- Responsibility: Shared models, protocol definitions and utilities used across components.

Changes must preserve explicit compatibility boundaries; shared code is not a separate data-owning service. Detailed contracts belong in the Roof Internal Library Specification.

9. Networking Specification

Networking principles apply across deployment profiles, while particular modes depend on the host and release. Appliance networking features must not become mandatory general-purpose-host behaviours.

9.1 Network Environments

Access-point mode belongs to the dedicated-appliance onboarding and local-connectivity design. It depends on suitable hardware and privileged network management and should be explicitly enabled only where appropriate.

Routed local networks provide the ordinary client-to-node path.

Direct Ethernet and operation through mesh networks or extenders depend on host and network capabilities.

These environments do not imply that Roof supplies mesh routing or guarantees compatibility with every topology.

9.2 Network Behaviour Principles

The following principles govern how the Roof system behaves across all supported network modes:

- **Local-First Operation**
  The system shall prioritise direct local communication between client devices and the Roof node whenever available.
  - Local connectivity is the primary and preferred operational path
  - Reduces dependency on external infrastructure
  - Preserves performance and data sovereignty

- **Independence from Internet Connectivity**

Core system functionality shall not require internet access when operating within a local environment.
    - Local file access and management must remain functional offline
    - Setup, control, and core operations must be achievable without external services

- **Graceful Degradation**
  The system shall maintain partial functionality under degraded network conditions.
    - Automatically fallback between network modes where applicable
    - Preserve data integrity during interruptions
    - Allow recovery and continuation of operations without system failure

10. Security and Data Privacy Model

This section defines the core security and data privacy principles of the Roof ecosystem. It establishes the foundational rules that govern how data is protected, accessed, and controlled across the system.

These are design requirements, not a security certification or a guarantee of the preview's protection.

Component specifications and security reviews must define threat models, trust boundaries, key management, authentication, authorisation, metadata handling and recovery.

Source publication alone does not validate those properties.

10.1 Core Principles

- **End-to-End Encryption by Design**
  All data transmitted between client devices and Roof nodes, as well as between Roof nodes, shall be designed to support end-to-end encryption.
    - Data must remain encrypted during transmission
    - Intermediary systems must not have access to decrypted payloads
    - Encryption is considered a baseline requirement, not an optional feature

- **No Central Data Ownership**
  The Roof ecosystem shall not introduce any centralised authority that owns, stores, or controls user data.
    - Data resides on user-operated Roof nodes
    - No central service shall act as a persistent data store for user content
    - System architecture must avoid reliance on third-party data custody

- **Relay as Blind Routing Only**
  Any intermediary network coordination or relay component shall operate strictly as a routing layer.
    - Relays must not inspect, decrypt, or persist user data
    - Their role is limited to connection facilitation and packet forwarding

    - Relays must not be treated as trusted data-processing entities

- **User Sovereignty and Control**
  Users shall retain full ownership and control over their data and its access.
    - Users define access permissions and data-sharing decisions
    - No system component shall override user-defined control without explicit action
    - Data access must be transparent and user-governed

11. System Constraints & Assumptions

This section defines the key constraints and operating assumptions under which the Roof system is designed to function. These conditions inform architectural decisions, system behaviour expectations, and overall design trade-offs.

11.1 System Constraints

The following constraints guide the target architecture; the current appliance implementation still contains dependencies to be removed or isolated:

- **Host Resources and Privilege Boundaries**
  Operate efficiently within supported host resources and privileges. Ordinary software operation must not assume control of a dedicated operating system.
    - Limited CPU, memory, and storage capacity must be expected
    - Performance characteristics may vary depending on hardware configuration
    - System design must prioritise efficiency and lightweight operation

- **Decentralised and Localised Infrastructure**
  The system must function without reliance on centralised infrastructure for core operations.
    - No mandatory third-party cloud service for core local operations
    - Core functionality must remain available within local environments
    - Remote paths may depend on external coordination services; their failure must not disable core local operation

- **Network Variability**
  The system must operate under varying and unpredictable network conditions.
    - Users may experience unstable or low-quality connections
    - Network configurations may differ significantly across environments
    - Connectivity interruptions must be expected as a normal condition

11.2 System Assumptions

The following assumptions define expected conditions about users and environments:

- **Intermittent Connectivity**
  It is assumed that continuous internet connectivity is not guaranteed.
    - Users may operate fully or partially offline

    - Remote connectivity may not always be available
    - System must tolerate temporary disconnections
- **Non-Technical Users**
  Accessible operation for non-technical users remains an end-user goal. Developer Preview 0.1.0 instead assumes technical source-build and evaluation skills.
    - Setup and operation must be intuitive and guided
    - System must minimise required technical intervention
    - Error states should be recoverable without advanced knowledge

- **User-Managed Infrastructure**
  Users are responsible for their chosen host, storage, credentials and backups, whether Roof ultimately runs on an everyday computer or a dedicated device.
    - Users manage power, physical environment, and basic connectivity
    - Hardware availability and uptime are user-dependent
    - System must accommodate variability in user-managed environments

12. Risks & Limitations

User-managed infrastructure carries availability, security and recovery risks. The current Engine also includes privileged Linux and appliance-management behaviour.

Developer Preview 0.1.0 does not establish safe installation on ordinary computers; technical evaluation must follow the release's documented environment and isolation requirements.

12.1 Identified Risks

- **Power Outages**
  The Roof node operates within a user-managed physical environment and is dependent on continuous power availability.
    - Loss of power will result in temporary system unavailability
    - Data access and node communication will be interrupted during outages

- **Network Dependency**
  System functionality is influenced by the availability and quality of network connectivity.
    - Local operations depend on local network conditions
    - Remote access depends on external network availability
    - Network instability may impact performance and accessibility

- **Hardware Failure**
  The system relies on consumer-grade hardware which may be subject to failure over time.
    - Storage, memory, or device-level failures may impact system operation
    - Data availability may be affected depending on failure conditions
    - Hardware reliability is not guaranteed across all environments

- **User Misconfiguration**
  As a user-operated system, incorrect configuration may impact functionality, security, or accessibility.
    - Improper network or credential configuration may restrict access
    - Mismanagement of settings may lead to degraded performance or exposure risks
    - Recovery may require user intervention

12.2 System Limitations

The following limitations are inherent to the architectural approach of the Roof system:

- **Availability Bound to User Environment**
  System uptime is dependent on the user's local environment.
    - Unlike centralised systems, no guaranteed uptime SLA is provided
    - Availability varies based on power, network, and hardware conditions

- **Absence of Centralised Recovery Mechanisms**
  The system does not rely on centralised backup or recovery infrastructure by default.
    - Data recovery is dependent on user-managed strategies
    - Loss of local data without backup may be irreversible

- **Performance Variability**
  System performance may vary depending on hardware capability and network conditions.
    - No uniform performance baseline across all deployments
    - Resource constraints may impact responsiveness under load

13. Roadmap & Future Phases

The public development roadmap owns changing schedules, milestones and acceptance criteria.

This root records the direction and the distinction between historical phases, source publication and future runtime readiness.

13.1 Current Direction

The current sequence is source publication, hardware-dependency assessment, generic-host proof of concept, independent-runtime validation, software-first deployment and broader platform evaluation.

Dedicated hardware remains an optional future deployment profile. Consult Roof Open-Source Public Development Roadmap for the maintained schedule.

13.2 Preview Scope
Developer Preview 0.1.0 is planned for 16 November 2026. Its scope is approved source, applicable licences and third-party notices, architecture and build guidance, documented dependencies and limitations, contribution channels and a security-reporting route.

It excludes prebuilt installers and supported general-purpose-device deployment.

13.3 Entry Criteria
Before a milestone begins, its owner must define the component scope, target environment, dependencies, evaluation conditions and acceptance criteria.

A milestone's inclusion in the roadmap is not evidence that it has begun or passed.

13.4 Exit Criteria
A milestone is achieved only when its stated acceptance criteria are met and verified capabilities, remaining limitations and supported environments are documented.

Source-publication readiness does not establish independent-runtime or end-user readiness.

13.5 Phase Transition Rules
Progression is evidence-based. Dates may change with engineering results, security findings and support capacity.

Changes belong in the maintained roadmap and release notes; this root is revised when project purpose, architectural boundaries or material historical context changes.

13.6 Future Considerations
Future work may include additional operating systems, richer application integration, broader federation and Buddy System research.

These are directions for evaluation, not commitments of the initial preview.

14. Compliance & Legal
This section defines project responsibilities and boundaries. It makes no claim of legal certification or universal regulatory compliance.

Applicable obligations depend on the deployment, participants and data involved; detailed policies and release terms are maintained separately.

14.1 Compliance Scope
DFLab maintains project specifications and release materials; operators and application developers remain responsible for their respective deployments and data practices.

Component and policy documents should identify relevant data handling and operational responsibilities.

14.2 Data Privacy Compliance

Privacy principles require purposeful data handling, explicit access control and clear disclosures about storage, sharing and metadata.

The Australian Privacy Principles, including APP 11, remain a relevant reference for assessing applicable obligations; this document does not determine legal applicability or assert compliance.

14.3 Jurisdiction Considerations

Deployment location, service operation and data-sharing arrangements can cross jurisdictions. Local-first architecture is not by itself proof of data residency or compliance.

Deployment-specific assessments belong outside this conceptual specification.

14.4 Roles & Responsibilities

DFLab maintains design and release guidance.

Users manage their hosts, authorisations and backups.

Contributors and application developers document dependencies and data practices, respect component boundaries and follow applicable contribution and release requirements.

14.5 Legal Boundaries of the System

The intended architecture avoids central custody of user content. This does not establish a legal exemption, transfer every responsibility to users or constitute a limitation-of-liability agreement.

Applicable licences, third-party notices and release terms must accompany published materials; this specification does not grant rights over unpublished code.

14.6 Future Regulatory Alignment

Review policies and component requirements as deployment models, data practices and applicable obligations evolve.

Record material changes in the appropriate policy, release or component documentation.

15. Human Interventions and Interactions

Content Filtering and Moderation

User-operated storage does not imply central inspection of private content.

Shared applications and publicly operated services need context-specific access, abuse-handling and moderation policies without weakening the core privacy boundaries.

User Verification and Authentication

Authentication and authorisation must establish who may access a node or operation. They do not automatically verify a person's real-world identity.

Component specifications define trust establishment, credential protection and recovery.

Reporting and Response Mechanisms

Release documentation should provide a private security-reporting route and distinguish vulnerability reports from ordinary defects and community-conduct concerns. Response ownership and processes belong in maintained project policies.

Data Privacy and Consent

Sharing and application access require explicit, understandable user decisions.

Permission scope, revocation and retention behaviour must be documented and validated for supported workflows.

Community Guidelines

Open development should use published contribution and conduct guidelines with clear maintainer responsibilities.

Broader community governance remains a future direction rather than an established governance model.

Legal Compliance

User interactions and application services remain subject to the responsibility boundaries in Section 14 and applicable project policies.

Decentralisation does not remove those responsibilities.

Technological Safeguards Against Modern Innovations

Applications, including AI integrations, must not treat access to personal Roof data as permission to use it for model training.

Data use, export and training require explicit authorisation and documented handling boundaries.

Application isolation and enforcement belong in component specifications.

Appendices

Glossary

- Roof node: a user-operated Roof environment, historically a dedicated appliance and intended to become a software runtime on supported hosts.
- Personal Internet: Roof's long-term vision of interconnected personal computing and data environments.
- Local-first: core local operations take priority over externally coordinated paths.

Acronyms

- AP: Access Point.
- FDB: Federated Database.
- RApp: Roof Native App.
- IA/BT/PA/BA/DA/UA: historical Internal, Beta Test, Public, Business, Developer and Universal Availability phase labels. These labels are not current release-status declarations.

Diagrams

Section 7.3.7 records the root-level component relationships. Detailed deployment, protocol and sequence diagrams belong in the respective component specifications.

References and Document History

The Roof Model - Open Specification is the inception document (2024), recording the FDB motivation, Raspberry Pi Zero 2 W concept and founding principles.

The superseded Roof Project Specification records the later Raspberry Pi 5 direction. Both remain historical references, not current support definitions.

This revision updates the Roof Project Root Specification last dated 23 April 2026, preserves its architectural lineage, and records the October 2026 open-source decision and the current software-first direction as of 9 November 2026.

Component references are listed in Section 7.2.

The Roof Open-Source Public Development Roadmap maintains targets; repository documentation and release notes establish actual source availability, supported builds, validated functionality and known limitations.